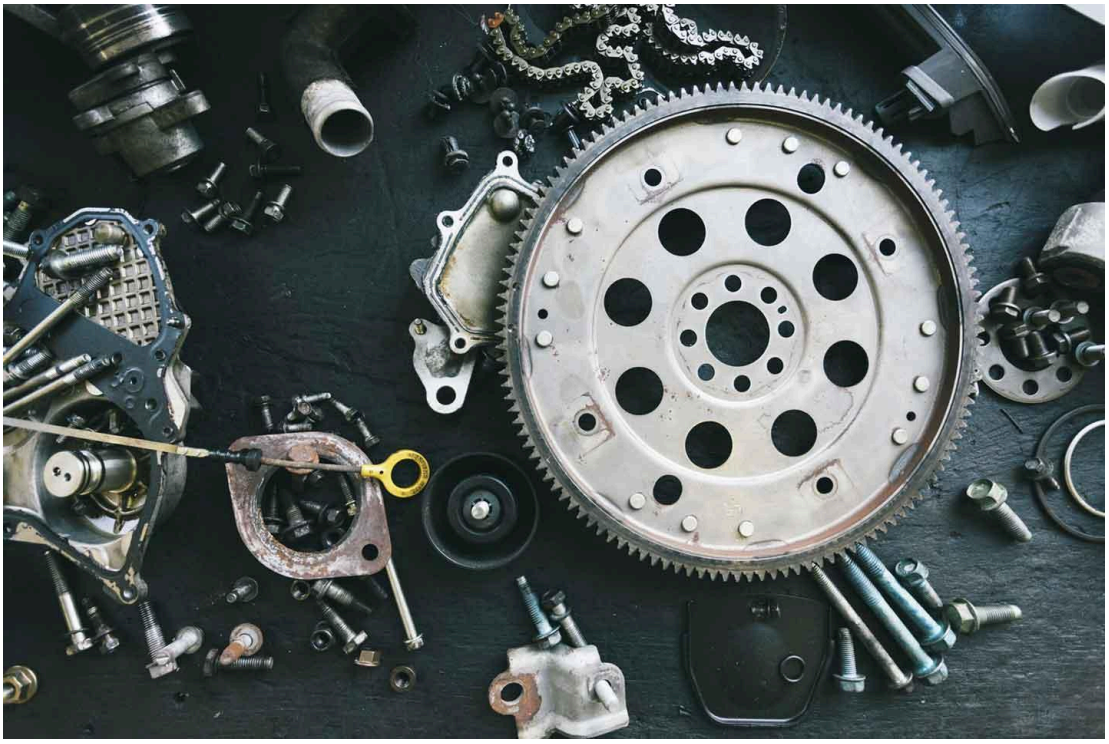
Tema 6

Relaciones Uno a Muchos



Francisco Belda

ÍNDICE

Teoría. Relaciones Uno a Muchos **3**
Paso a Paso: Implementación Práctica **4**
Creación de la entidad Curso (Lado Propietario con @ManyToOne) 4
Actualización de la entidad Profesor (Lado Inverso con @OneToMany) 5
Práctica Guiada: Repositorio y Pruebas **9**
Creación del CursoRepository 9
Verificación en el Inicializador de Datos 9
Ejercicios **10**
Conceptos Clave Recordados 10
Ejercicios 10

Teoría. Relaciones Uno a Muchos

Una relación Uno a Muchos significa que una instancia de la Entidad A puede estar asociada con múltiples instancias de la Entidad B, mientras que cada instancia de la Entidad B está asociada exactamente con una instancia de la Entidad A.

- **Lado Propietario (@ManyToOne)**: Es la entidad que contiene físicamente la clave ajena (Foreign Key) en su tabla en la base de datos. Siempre lleva la anotación @JoinColumn.
- **Lado Inverso (@OneToMany)**: Es la entidad que posee la colección (por ejemplo, un List<Curso>). Se configura utilizando el atributo mappedBy para indicarle a Hibernate que el mapeo físico lo gestiona el otro extremo.

Paso a Paso: Implementación Práctica

Vamos a relacionar nuestras entidades **Profesor** y **Curso**, de modo que un profesor pueda tener una lista de cursos asignados.

Creación de la entidad Curso (Lado Propietario con @ManyToOne)

Crea la clase **Curso.java** dentro del paquete **model**:

```
package com.educacion.demospring.model;


import jakarta.persistence.*;
        import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.ToString;


@Entity
@Table(name = "cursos")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Curso {


    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;


    @Column(name = "nombre_curso", nullable = false, length = 100)
    private String nombreCurso;
```

```
    @Column(name = "creditos")
    private int creditos;

    // Relación Muchos a Uno: Muchos cursos pertenecen a un profesor
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "profesor_id", nullable = false)
      @ToString.Exclude // Evita bucles infinitos en el toString generado por Lombok
@JsonIgnore
    private Profesor profesor;

    // Constructor auxiliar sin ID
    public Curso(String nombreCurso, int creditos, Profesor profesor) {
        this.nombreCurso = nombreCurso;
        this.creditos = creditos;
        this.profesor = profesor;
    }
}
```

```
@ManyToOne(fetch = FetchType.LAZY)
```

El parámetro **fetch** define CUÁNDO se consultará la base de datos para traer la información del objeto relacionado:

LAZY (Carga perezosa / bajo demanda):
Cuando consultas un Curso en la base de datos (findById o findAll), JPA NO hace un JOIN ni busca los datos del Profesor inmediatamente. En su lugar, coloca un objeto proxy (fantasma) en el atributo profesor.

La consulta SQL para traer los datos del Profesor solo se ejecutará cuando hagas uso explícito de ese atributo en el código Java (por ejemplo, al hacer curso.getProfesor().getNombre()).

Actualización de la entidad Profesor (Lado Inverso con @OneToMany)

Modifica la clase **Profesor.java** para incluir la lista de cursos y la relación inversa:

```
package com.educacion.demospring.model;


import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.ToString;
import java.util.ArrayList;
import java.util.List;


@Entity
@Table(name = "profesores")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Profesor {


    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;


    @Column(name = "nombre", nullable = false, length = 50)
    private String nombre;


    @Column(name = "especialidad", nullable = false, length = 50)
    private String especialidad;
```

```
    // Relación Uno a Uno previa con Oficina (si la tienes implementada)
    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    @JoinColumn(name = "oficina_id", referencedColumnName = "id")
    private Oficina oficina;


    // Relación Uno a Muchos: Un profesor imparte muchos cursos
    @OneToMany(mappedBy = "profesor", cascade = CascadeType.ALL, orphanRemoval = true)
    @ToString.Exclude // Previene recursividad en toString de Lombok
@JsonIgnore // EVITA EL BUCLE INFINITO (StackOverflowError) al serializar JSON
    private List<Curso> cursos = new ArrayList<>();
    // Constructor auxiliar sin ID y sin cursos
    public Profesor(String nombre, String especialidad, Oficina oficina) {
        this.nombre = nombre;
        this.especialidad = especialidad;
        this.oficina = oficina;
    }
    // Método de utilidad para sincronizar ambos lados de la relación bidireccional
    public void agregarCurso(Curso curso) {
        cursos.add(curso);
        curso.setProfesor(this);
    }
// Setter personalizado que intercepta la deserialización JSON de Postman
public void setCursos(List<Curso> cursos) {
    this.cursos = cursos;
    if (cursos != null) {
```

```
        for (Curso curso : cursos) {
            curso.setProfesor(this);
        }
    }
}
}
```

Cuando envías la información del profesor en formato JSON, Jackson (el serializador de Spring) deserializa el objeto Profesor y su lista de cursos. Sin embargo, como el campo profesor está en el lado de Curso (@ManyToOne), por defecto llegará como nulo a la base de datos, lo que provocará un error de clave ajena (not-null property references a null or transient value) si tu columna profesor_id es obligatoria.

Para solucionar esto de forma automática y limpia, añadimos este setter personalizado **setCursos** en la entidad **Profesor.java**. Así, cada vez que Postman envíe la lista de cursos, se asociarán automáticamente al profesor.
Secuencia de inserción de JSON:

1. Spring Boot intercepta la petición HTTP.
2. Jackson crea una instancia vacía de la entidad usando el constructor por defecto sin parámetros.
3. Jackson asigna los campos simples llamando a los setters o por reflexión.
4. Jackson deserializa la lista de cursos creando cada objeto Curso individual.
5. Jackson invoca a profesor.setCursos(listaDeCursos) con lo que la propiedad profesor dentro de cada objeto Curso deja de ser null y se asigna al Profesor.

Modificamos el PUT y PATCH del Controller:

PUT:

Añadimos la lista:

```
profesorExistente.getCursos().clear();
if (profesorDatos.getCursos()!=null){
    for (Curso curso: profesorDatos.getCursos()){
        curso.setProfesor(profesorExistente);
        profesorExistente.getCursos().add(curso);
    }
}
```

PATCH:

Añadimos el case cursos:

```
case "cursos":

    // Verificación de tipo: Se valida que el valor entrante bajo la clave "cursos" sea efectivamente una lista JSON.

   if (valor instanceof List) {


        // Casting seguro: Se realiza el cast de 'valor' a una lista de mapas (donde cada mapa es el JSON de un curso).

// Sirve para convertir la variable genérica valor

// (que llega como Object desde el mapa de campos del PATCH) a un tipo estructurado

// List<Map<String, Object>> para poder iterar cada curso,

// utilizando @SuppressWarnings("unchecked") para silenciar la advertencia de compilación

// sobre tipos genéricos en Java.

       @SuppressWarnings("unchecked")

                List<Map<String, Object>> listaCursosMap = (List<Map<String, Object>>) valor;


          // Limpieza de colección previa: Se vacía la lista actual de cursos del profesor.

         // Al usar 'orphanRemoval = true' en JPA, las referencias desasociadas se eliminarán automáticamente de la BD al guardar.

       profesorExistente.getCursos().clear();


        // Iteración sobre los nuevos cursos: Se recorre cada curso recibido dentro del array JSON.

       for (Map<String, Object> mapAsig : listaCursosMap) {


            // Instanciación de un nuevo objeto entidad Curso

           Curso asig = new Curso();
```

```
                // Asignación del ID (si existe): Mapea el identificador existente convirtiendo el valor numérico a Long.
            if (mapAsig.containsKey("id") && mapAsig.get("id") != null) {
                                                asig.setId(((Number) mapAsig.get("id")).longValue());
            }
            // Asignación del nombre: Se obtiene el texto de la clave "nombre" y se convierte a String.
            asig.setNombre((String) mapAsig.get("nombre"));

            // Asignación de créditos: Comprueba si 'creditos' es un número para evitar ClassCastException y lo asigna como Integer.
            if (mapAsig.get("creditos") instanceof Number) {
                                            asig.setCreditos(((Number) mapAsig.get("creditos")).intValue());
            }

                // Relación bidireccional (Clave Foránea): Se vincula el curso con la instancia del profesor existente.
            // Es indispensable para que 'profesor_id' no quede registrado como NULL en la base de datos.
            asig.setProfesor(profesorExistente);

              // Persistencia en memoria: Se añade el curso ya configurado a la colección de la entidad Profesor.
            profesorExistente.getCursos().add(asig);
        }
    }
   break;
```

Práctica Guiada: Repositorio y Pruebas

Creación del CursoRepository

Crea la interfaz **CursoRepository.java** dentro del paquete **repository**:

```
package com.educacion.demospring.repository;

import com.educacion.demospring.model.Curso;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;


@Repository

public interface CursoRepository extends JpaRepository<Curso,
Long> {}
```

Verificación en el Inicializador de Datos

Prueba la inserción en cascada desde el **profesor** hacia sus **cursos** usando el método de utilidad:

```
Profesor profesor = new Profesor("Carlos Soria", "Desarrollo
Web", oficina);

Curso curso1 = new Curso("Programación en TypeScript", 6,
profesor);

Curso curso2 = new Curso("Frameworks Avanzados Angular", 8,
profesor);

// Añadimos los cursos al profesor utilizando el método helper

profesor.agregarCurso(curso1);

profesor.agregarCurso(curso2);

profesorRepository.save(profesor);

System.out.println(">>> Profesor y sus cursos guardados
correctamente en cascada.");
```

Ejercicios

Conceptos Clave Recordados

@ManyToOne define el lado propietario que almacena físicamente la clave foránea en la tabla.

@OneToMany(mappedBy = "...") define el lado inverso que apunta al atributo de la entidad propietaria.

El uso de **@ToString.Exclude** con Lombok es indispensable para evitar bucles infinitos al imprimir entidades relacionadas bidireccionalmente.

Ejercicios

1. Crear la **entidad ContactoAuxiliar**, que tendrá **nombre, telefono** y **email.**
2. Crear la **relación 1aN** de **Estudiante** con **ContactoAuxiliar.**
3. Probar a **insertar** y **ver** los resultados con **Postman.**